AI 极客挑战赛 · 题目

赛题总览

编号	赛题名称
T1	名老中医传承工作室全流程智能化管理平台开发
T2	基于大模型的自动本体构建与知识入库原型
T3	Claw产品 - NL2UI 对话实现
T4	爆款视频要素分析与复刻策略工具
T5	智能报告写作平台——知识库生成 × 以稿写稿
T6	WorkClaw - 云端 OpenClaw 办公助手
T7	Claworks - 高并发 WebSocket 集成与 Agent 通信

总体要求

- **各题目可单人参赛，也可组队参赛（组队不可超过3人）**
- **必须交付可运行的实物**，不接受纯 PPT 演示
- **必须提交 AI 工具使用日志**（使用了哪些工具、解决了哪些问题），无日志不得参加答辩
- 技术路线不限：可使用 Cursor、Claude Code、GitHub Copilot 等任意 AI 编程工具
- 答卷请整理至一份压缩包，**命名“姓名-题目”**，材料需整理清晰，写明必要的指引

T1 | 名老中医传承工作室全流程智能化管理平台开发

基础信息

- **对应业务场景：** 3.1 构建智能化传承工作室管理平台

分工角色要求

- **全栈开发负责人：** 统筹系统架构设计，核心业务模块开发，AI 辅助编程工具落地与全流程管控
- **数据与检索模块负责人：** 负责资源数据模型设计、多类型资源智能检索功能实现、数据库设计
- **前端与权限负责人：** 负责前端页面开发、用户交互逻辑实现、多角色权限体系设计

核心任务要求 赛题一 名老中医传承工作室全流程智能化管理平台开发

1. 完成工作室管理平台 5 大核心模块开发，覆盖基础信息管理、学术活动管理、资源管理（医案 / 影像 / 论文）、学员培养管理、绩效考核管理全流程业务需求。
2. 实现医案、影像、论文等多类型资源的统一结构化存储与智能检索，支持按医师、病症、方剂、关键词等多维度精准检索与模糊匹配。
3. 完成分级权限体系设计，区分工作室负责人、传承医师、学员、系统管理员 4 类角色的操作权限与数据访问范围。
4. 全程使用 AI 辅助编程工具完成核心代码开发，提交完整的 AI 辅助编程过程记录（含 prompt 设计、代码生成迭代记录、工具应用效率说明）。

交付物清单

- 可正常运行的系统完整代码包、部署文档、数据库初始化脚本
- 系统全功能演示视频（5 分钟以内）
- AI 辅助编程应用说明文档
- 系统架构设计与业务需求匹配说明书

评定标准（满分 100 分）

评审维度	权重	评审细则
业务需求匹配度	40%	核心模块完整度、功能与业务需求的贴合度，每缺 1 个核心模块扣 8 分，功能不符合业务逻辑酌情扣分
代码质量与完成度	25%	代码可运行性、规范性、可维护性、BUG 率，无法正常运行扣 20 分，代码不规范、存在安全漏洞酌情扣分
AI 辅助编程应用能力	20%	prompt 设计合理性、工具使用效率、AI 生成代码的优化与落地能力，无过程记录不得分
方案创新性与可用性	10%	检索效率、交互体验、架构扩展性的优化设计
合规与文档完整性	5%	文档齐全度、权限设计合规性、数据安全防护能力

保密要求

赛题配套的业务原型、工作室参考数据仅限参赛队伍内部使用，严禁对外传播；代码成果知识产权归主办方所有，未经授权不得对外开源或泄露。

T2｜基于大模型的自动本体构建与知识入库原型

背景

在构建企业级决策智能平台时，本体建模往往是最耗时的人工环节。DIP 产品线核心能力之一是实现数据到知识的自动化沉淀。本题要求利用大模型的语义理解能力，构建一个从零散 Excel 业务数据到结构化本体体系的自动转换原型，实现上传即建模，建模即入库。

核心需求

1. **多源语义解析与本体提取：**
   - 支持上传单个或多个 Excel 文件（如：物流产品表、订单明细、客户名录等）。
   - 利用大模型自动识别表头语义，推断本体及其属性，如从 "订单表" 中识别出 "订单号"、"下单时间" 等属性。
2. **跨表关系发现：**
   - 上传多个 Excel 时，AI 需自动分析不同表之间的关联逻辑（如：订单表中的 "客户 ID" 与客户表的关系），构建本体间的关联关系。
3. **人机协同确认环路：**
   - 提供可视化界面展示 AI 预构建的本体和关系拓扑。
   - 支持人工对 AI 识别结果进行修正、确认或驳回。只有确认后的模型才允许正式入库。
4. **数据自动化实例映射：**
   - 确认本体结构后，Excel 中的每一行数据需自动转化为本体对象并关联保存。
5. **知识管理后台：**
   - 提供本体一览（Schema 查看）、关系拓扑一览、以及对本体对象的增删改查功能。
6. **业务场景模拟：**
   - 围绕物流行业：包含产品、订单、客户、销售员、合同、报价单等核心要素。
   - 测试数据：参赛者需利用 LLM 自行生成符合逻辑的模拟 Excel 数据作为演示样例。

交付要求

- 完整可运行的原型系统：需包含前端交互界面（展示本体图谱）与后端逻辑。
- 自动化建模演示视频：演示从上传原始 Excel 到生成关系图谱的全过程。
- 实验数据集：提交由 AI 生成的物流业务模拟 Excel 文件包。
- AI 工具使用日志：记录如何利用 Cursor/Claude 等工具辅助开发，以及 Prompt 调优记录。

评分标准

维度	权重	评分要点
建模准确性	40%	AI 对 Excel 语义理解的准确度，关系发现是否符合业务逻辑
产品闭环程度	30%	从 "解析" 到 "确认" 再到 "数据入库" 的全流程是否流畅、闭环
交互设计	20%	本体拓扑图的可视化效果，人工确认环节的易用性
技术架构	10%	Prompt 设计的健壮性，是否考虑了处理大批量数据的性能

T3 | Claw 产品 - NL2UI 对话实现

背景

在当前 Claw 产品的用户对话流程中，当 AI 需要用户进一步确认信息、补充输入内容或进行选项选择时，通常通过自然语言文本向用户发起交互请求。例如让用户回复 "是 / 否"、输入参数，或在多轮对话中补充必要信息。这种纯文本交互方式在实际使用中存在明显问题：

- 用户需要理解 AI 的文字提示后再进行输入，操作效率较低
- 文本输入容易产生歧义，影响 AI 对用户意图的理解
- 在确认、选择、填写等结构化场景下，文本交互体验较差
- 多轮补充信息过程较长，影响整体任务完成效率

随着对话式 AI 向任务型 Agent 和业务流程协同演进，仅依赖文本交互已经难以满足复杂业务场景需求。因此需要在 Claw 产品中引入 NL2UI（Natural Language to UI）能力。NL2UI 的核心思想是：当 AI 在对话过程中需要用户参与操作时，不再只返回文本提示，而是输出结构化 UI 描述（UI Schema）。前端根据该 Schema 自动渲染为可视化交互组件（如确认卡片、表单、选项按钮等），用户直接通过界面完成操作。用户操作结果再以结构化数据形式回传给 AI，使对话能够继续推进任务流程，从而将传统文本确认交互升级为可视化交互界面，显著提升用户体验与操作效率。本题要求构建一个 NL2UI Skill + MCP 能力原型，验证 AI 在对话过程中动态生成 UI 并与用户交互的可行性。可以在 copaw、openclaw、HiClaw 等开源平台的基础上实现并封装成标准的插件供 agent、claw 系列平台复用。

核心需求

1. **NL2UI 结构化输出：** 在用户对话过程中，当 AI 需要用户确认或补充信息时，应能够生成标准化 UI Schema，而不是继续使用纯文本提问。UI Schema 至少支持以下类型：信息确认（确认卡片）、选项选择（单选 / 多选）、参数填写（表单输入）、操作确认（提交 / 取消）

2. **前端 UI 自动渲染：** 系统需根据 AI 返回的 UI Schema 自动生成可视化界面组件，例如：确认卡片、单选 / 多选按钮、表单输入框、参数填写组件、操作按钮。用户无需通过文本输入即可完成操作。

3. **手机渠道可交互：** 用户需能在手机渠道收到并操作上述可交互 UI。优先支持在钉钉、飞书等机器人平台对话中展示并操作 UI。若所选机器人平台不支持富交互 / 卡片等能力，则需额外提供 H5 端，使手机端用户通过 H5 打开链接即可完成确认、选择、表单填写等操作，并将结果回传至对话流程。

4. **用户操作结果回传：** 用户在 UI 上的操作（点击、选择、填写等）需要被系统捕获，并以结构化数据形式回传给 AI，使 AI 能够继续完成后续对话或任务流程。

5. **对话流程无缝衔接：** UI 出现时应说明当前任务节点；用户操作后 AI 能继续推进对话；对话历史中可记录 UI 交互结果。

6. **MCP 能力集成（可选加分）：** 可结合 MCP 工具能力，使 AI 在完成 UI 交互后能够继续执行任务，例如：调用外部工具查询数据、更新系统状态、生成结果或报告。实现 对话 → UI 交互 → 工具调用 → 结果输出 的完整流程。

交付要求

- 可运行的 Web 原型系统
- 手机渠道可收到可交互的 UI：在钉钉、飞书等机器人对话中可展示并操作 UI；若平台不支持，则需提供 H5 端，使手机端用户能打开链接完成确认 / 选择 / 表单等操作并回传结果
- 至少演示 3 个 NL2UI 场景（如信息确认、选项选择、表单输入）
- 对话流程中能够动态生成 UI 并完成交互
- 用户操作结果能够回传并继续对话流程
- AI 工具使用日志

建议演示场景示例：

- AI 请求用户确认任务参数
- AI 提供选项让用户选择处理策略
- AI 通过表单收集结构化信息

评分标准

维度	权重	评分要点
产品价值	35%	NL2UI 是否显著提升对话交互体验
技术实现	30%	UI Schema 设计与前端渲染是否稳定
AI 使用深度	20%	AI 是否能够动态生成 UI 并参与流程决策
交互体验	15%	UI 与对话结合是否自然、易用

T4 | 爆款视频要素分析与复刻策略工具

背景

优雅产品4月核心需求：**爆款复刻功能**，对爆款视频的脚本、台词、画面、要素进行多维度分析，自动生成复刻策略建议。本题要求构建该功能的独立可用原型，重点验证"AI分析爆款要素→生成可操作复刻建议"这一核心能力。

核心需求

1. **内容输入**：支持输入视频公开链接（解析互动数据）、上传视频文件（MP4，30分钟以内），或粘贴脚本文本
2. **爆款要素拆解**：
   - 脚本结构分析：识别开头钩子/冲突/转折/结尾行动号召
   - 台词风格分析：语气特征、句式规律、高频关键词
   - 情绪节奏分析：推测铺垫→高潮→共鸣的情绪曲线
   - 核心传播要素：提炼3-5个最能引发传播的关键要素
   - 目标受众推断：基于内容特征推测受众画像
3. **复刻策略报告**：输出可操作的复刻建议，包含可复用脚本框架、关键要素具体化建议、风险规避提示
4. **加分项**：支持同时输入2-3个视频，横向对比爆款要素差异

交付要求

- 可运行工具
- 至少3个真实爆款视频（不同类型：媒体类/营销类/知识类）的完整分析报告
- 复刻建议样例（展示建议的具体程度）
- Prompt设计文档 + AI工具使用日志

评分标准

维度	权重	评分要点
分析洞察深度	40%	结论是否有真正的洞察，而非泛泛而谈；是否发现了普通人不容易注意到的规律
复刻建议可操作性	30%	建议是否具体到"可以直接照着做"，而非停留在方向层面
技术完整性	20%	工具稳定运行，能处理不同类型的输入
呈现质量	10%	分析报告清晰易读，重点突出

T5｜智能报告写作平台——知识库生成 × 以稿写稿

背景

当前报告写作平台依赖闻海和互联网数据生成报告，存在两个核心痛点：**一是缺乏对内部知识资产的有效利用**（大量历史报告、政策文档、行业资料沉睡在文件夹里）；**二是"以稿写稿"完全依赖人工**（分析师参考旧报告改写新版本效率极低）。本题要求在报告写作平台基础上，构建"知识库驱动生成"与"以稿写稿"两大核心能力，交付可直接使用的完整系统。

核心需求

模块一：知识库构建与管理

- 支持批量导入文档（PDF/Word/TXT/Markdown），支持通过协议外挂知识库
- 按分类管理（政策法规 / 行业报告 / 历史报告 / 媒体资讯）
- 支持知识库内容增删改查

模块二：知识检索与引用（核心）

- 用户输入报告主题/提纲后，自动从知识库检索最相关内容
- 检索结果标注来源文档、页码/段落，支持溯源验证
- 支持手动补充检索条件或排除不相关内容

模块三：AI报告生成

- 基于知识库内容生成报告（知识库驱动模式）
- 支持仿写：参考用户上传报告的大纲与写作风格，生成新主题报告
- 大纲可保存复用
- 报告质量保障：事实性错误检测、引用准确性检查、知识库覆盖度分析

模块四：以稿写稿 / 智能改写（核心）

- 上传一篇或多篇参考报告，系统自动解析报告结构
- 四种改写模式：
  - **数据更新**：保持结构，替换旧数据为新数据
  - **视角调整**：同内容，改视角/立场/受众（如行业报告 → 领导简报）
  - **内容扩展**：基于原稿框架，AI补充新分析、新案例、新章节
  - **风格转换**：正式↔通俗，中文↔英文
- AI续写能力：在原稿基础上续写新章节，与原稿逻辑、数据、风格保持一致
- 修订模式视图：改写前后差异对比（新增/删除/修改一目了然）

模块五：版本管理与导出

- 保存多个改写版本，支持随时回溯对比
- 支持导出 Word / PDF

交付要求

- 可直接打开使用的完整 Web 系统（五个模块全部可用）
- 3份不同主题的完整报告生成演示（含引用来源标注）
- 3篇参考报告，分别演示4种改写模式
- 至少1个续写演示（从参考稿续写新章节）
- 改写前后对比展示（修订模式视图）
- 设计说明文档 + AI工具使用日志

评分标准

维度	权重	评分要点
知识检索准确性	25%	检索结果是否相关，引用溯源是否准确
报告生成质量	25%	报告结构是否专业，内容是否有深度，仿写风格还原度
改写质量	25%	改写是否保留原稿精华，四种模式是否有区分度
对比展示与版本管理	15%	差异是否直观，版本管理是否完善
系统完整性与体验	10%	全流程是否跑通，操作是否流畅

T6 | WorkClaw - 云端 OpenClaw 办公助手

一、背景

1.1 市场现状

OpenClaw 是 2026 年初爆火的开源 AI 智能体项目，GitHub 星标超 25 万，成为历史上增长最快的开源项目。国内大厂纷纷入局：腾讯推出 QClaw/WorkBuddy、阿里推出 CoPaw/HiClaw、小米推出 miclaw、华为推出小艺 Claw。

1.2 现有问题

当前 OpenClaw 类产品存在以下痛点：

- **部署门槛高：** 原生 OpenClaw 需要用户自行配置环境、安装依赖，普通用户难以上手
- **资源占用大：** 每个用户需要一台独立的电脑或服务器持续运行，硬件成本高
- **移动端缺失：** 现有产品多为桌面端，移动办公场景支持不足
- **多租户难题：** 如何让大量用户共享云端资源，同时保证隔离性和安全性

1.3 本题目标

构建一个**云端动态编排的 OpenClaw 办公助手**，实现：

- 用户在 H5 / 小程序中 "领养" 一只专属小龙虾（OpenClaw 实例）
- 系统自动在云端（腾讯云）创建容器实例并部署 OpenClaw
- 用户通过 APP 对话窗口与自己的云端 OpenClaw 交互
- 聚焦企业办公场景：文档处理、日程管理、消息处理

核心技术挑战：实现多用户动态资源编排，解决 OpenClaw 规模化部署难题。

二、核心需求

2.1 云端动态实例编排（核心难点）

用户点击 "领养小龙虾" 后，系统应能够：

- 自动调用腾讯云 API 创建容器实例
- 在容器中部署 OpenClaw 运行环境
- 建立用户 APP 与云端实例的通信连接
- 实现多用户资源隔离

技术要求：

指标	要求
实例创建时间	≤60 秒（首次领养）
实例运行模式	常驻运行，7×24 小时在线，随时可用
多用户隔离	每用户独立容器，数据不互通
用户规模	支持 100 用户以内

设计决策：采用常驻运行模式而非按需启停，确保用户随时可与助手交流，无需等待启动。成本约 86 元 / 用户 / 月，100 用户规模下月成本约 8600 元，体验优先。

架构示意

```
┌──────────────────────────────────────────────────────────────────────┐
│                                用户层                                 │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐                           │
│  │ 对话界面 │  │ 技能广场 │  │ 模板中心 │                           │
│  └──────────┘  └──────────┘  └──────────┘                           │
│                              H5 APP                                  │
└──────────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────┐
│                          控制平面（后端服务）                          │
│  ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐                │
│  │ 用户管理 │ │ 实例编排 │ │ 技能管理 │ │ 模板管理 │                │
│  └──────────┘ └──────────┘ └──────────┘ └──────────┘                │
│  ┌──────────┐ ┌──────────┐ ┌──────────────────────┐                │
│  │ 消息路由 │ │ 技能商店 │ │ Skills/MCP 仓库      │                │
│  └──────────┘ └──────────┘ └──────────────────────┘                │
└──────────────────────────────────────────────────────────────────────┘
                                  │
                                  │ 腾讯云 API
                                  ▼
┌──────────────────────────────────────────────────────────────────────┐
│                       云资源层（腾讯云容器实例）                       │
│  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────────┐   │
│  │ 用户A 邮件管家   │  │ 用户B 舆情专家   │  │ 用户C 自定义     │   │
│  │  ┌────┐ ┌────┐   │  │  ┌────┐ ┌────┐   │  │  ┌────┐ ┌────┐   │   │
│  │  │技能│ │技能│   │  │  │技能│ │技能│   │  │  │技能│ │技能│   │   │
│  │  └────┘ └────┘   │  │  └────┘ └────┘   │  │  └────┘ └────┘   │   │
│  └──────────────────┘  └──────────────────┘  └──────────────────┘   │
└──────────────────────────────────────────────────────────────────────┘
```

2.2 H5 对话界面

提供移动端友好的 H5 对话界面，用户可以：

- 通过自然语言与 OpenClaw 对话
- 上传文件让 AI 分析处理
- 查看 AI 任务执行状态和结果

界面要求：

- 响应式布局，适配手机屏幕
- 支持文字、文件、图片等多类型消息
- 实时显示 AI 处理状态（思考中、执行中、已完成）
- 支持对话历史查看

2.3 办公场景能力

聚焦企业办公场景，至少支持以下三类能力：

场景	功能	示例
文档处理	文档摘要、数据分析、报告生成	"帮我分析这份销售数据"
日程管理	日程查询、会议提醒、日程创建	"明天有什么安排？"
消息处理	邮件摘要、智能回复、消息筛选	"总结今天的未读邮件"

2.4 用户"领养"流程

用户注册/登录 → 点击"领养小龙虾" → 系统创建云端实例（显示孵化动画，≤60秒） → 实例就绪，进入对话界面 → 用户与专属 OpenClaw 对话 → 实例 7×24 常驻运行，用户随时可回来继续对话 → 对话记忆持续保存，支持长期使用

2.5 技能广场（Skills Marketplace）

构建开放的技能生态系统：

- **官方技能库：** 集成开源社区优秀 Skills 和 MCP 服务
- **用户共创：** 用户可将自己创建的优秀 Skills 发布到社区
- **技能安装：** 一键安装 / 卸载技能到自己的小龙虾
- **评分评论：** 社区对技能进行评价和反馈

技能分类：

分类	示例技能
办公效率	邮件处理、日程管理、会议纪要
数据分析	Excel 分析、可视化图表、报告生成
内容创作	文案写作、PPT 生成、社交媒体
专业领域	化学分析、法律咨询、财务审计
开发工具	代码审查、API 调试、文档生成

****MCP 集成**：**

支持的 MCP 服务示例：

```
├── @anthropic/mcp-server-filesystem  # 文件系统操作
├── @anthropic/mcp-server-github      # GitHub 集成
├── @anthropic/mcp-server-slack       # Slack 消息
├── @anthropic/mcp-server-google      # Google 服务
└── 社区贡献的 MCP 服务...
```

2.6 小龙虾模板套件

提供预配置的专业角色模板，用户可快速创建专属场景的小龙虾：

模板名称	预装技能	适用人群
📧 邮件管家大师	邮件摘要、智能回复、优先级分类、日程提取	商务人士
🧪 化学材料分析师	分子式解析、文献检索、实验数据分析、安全提示	科研人员
📊 舆情报告专家	热点监控、情感分析、报告生成、趋势预测	市场/公关
💼 HR 招聘助手	简历筛选、面试问题、候选人评估、offer 对比	人力资源
📝 自媒体运营官	选题策划、标题优化、多平台发布、数据复盘	内容创作者
💰 财务分析师	报表解读、预算分析、成本核算、风险提示	财务人员

自定义选配流程

1. 选择基础模板（或从零开始）
   ▼
2. 进入技能广场，浏览可用技能
   ▼
3. 勾选需要的技能（可多选）
   ▼
4. 配置技能参数（如邮箱账号、API密钥等）
   ▼
5. 确认创建，生成专属小龙虾
   ▼
6. 后续可随时添加/移除技能

2.7 实例管理与运维

- **常驻运行：** 实例 7×24 小时在线，用户随时可用
- **健康检查：** 定期检测实例状态，异常自动重启
- **日志监控：** 记录实例运行状态、对话日志
- **容量规划：** 支持 100 用户规模，预留扩展能力

三、技术方案

层级	技术方案
前端	Vue3 + Vant（H5 移动端组件库）
后端	Python + FastAPI
通信	WebSocket（实时对话）
云平台	腾讯云容器实例（TCI）
数据库	PostgreSQL / MySQL
Agent	OpenClaw + 自定义 Skills
大模型	Kimi API / DeepSeek API

四、交付要求

- 可运行的 H5 原型系统
- 云端动态编排能力：用户领养时自动创建腾讯云容器实例
- 多用户隔离：每用户独立 OpenClaw 实例
- 技能广场：展示可用技能列表、支持安装 / 卸载技能
- 模板套件：至少提供 3 个预设模板（如邮件管家、舆情专家等）
- 自定义选配：创建时可勾选技能组合
- 至少演示 3 个办公场景（文档分析、日程查询、邮件处理）
- 完整的 选模板→领养→配置技能→对话 流程
- 技术架构文档与答辩 PPT

五、评分标准

维度	权重	评分要点
技术实现	35%	云端动态编排是否稳定、技能安装 / 卸载是否可靠、多用户隔离是否有效
产品体验	25%	模板选配是否顺畅、技能广场是否易用、对话交互是否自然
生态设计	20%	技能广场是否开放、用户共创机制是否完善、模板是否丰富实用
架构设计	10%	技术架构是否合理、可扩展性如何
创新性	10%	是否有独特的产品或技术亮点

六、成本估算

配置	单价	单用户/月	100用户/月
1核2G	≈0.12元/小时	≈86元	≈8,600元
2核4G	≈0.24元/小时	≈173元	≈17,300元

成本优化策略（后续可选）：

- 包年包月：长期用户转为包月实例，成本降低 30-50%
- 抢占式实例：非核心场景使用抢占式实例，成本降低 70%
- 资源复用：低活跃用户可考虑共享实例（牺牲部分隔离性）

七、技术风险与应对

风险	概率	应对方案
腾讯云 API 不熟悉	高	Day1 专门研究文档，准备备用方案
容器启动慢	中	实例池预热 + 镜像优化
OpenClaw 镜像问题	中	提前构建测试，准备 Mock 方案
网络连通性	中	使用 VPC 内网，配置安全组

降级方案： 若动态编排在 Day5 仍未调通，立即切换到简化方案：预先创建 2 台轻量服务器，手动部署 OpenClaw；演示时强调 "生产环境会用动态编排"；答辩时展示完整架构设计。

附录

参考资料：

- GitHub - openclaw/openclaw: Your own personal AI assistant. Any OS. Any Platform. The lobster way. 🦞 · GitHub
- 云产品文档中心_云产品帮助中心
- FastAPI

竞品参考：

产品	厂商	特点
QClaw	腾讯	本地部署、微信远程操控
WorkBuddy	腾讯	免部署、兼容 OpenClaw 技能
StepClaw	阶跃星辰	云端 7×24 在线
miclaw	小米	系统级、IoT 联动

T7 | Claworks - 高并发 WebSocket 集成与 Agent 通信

一、背景

市场现状

OpenClaw 是 2026 年初爆火的开源 AI 智能体项目，GitHub 星标超 25 万。国内大厂纷纷推出云端化、移动化产品，如腾讯 QClaw/WorkBuddy、阿里 CoPaw、小米 miclaw 等。WorkClaw 作为一款云端 OpenClaw 办公助手，已在多用户动态编排、模板选配、技能广场等方面取得初步成果。

现有问题

随着用户量增长和办公场景复杂化，现有 WorkClaw 原型暴露出以下性能与架构瓶颈：

1. **WebSocket 连接高并发压力**
   每个用户独占一个容器实例，且需保持长连接进行实时对话。当用户数达到 100+ 时，后端 WebSocket 服务面临连接数、消息吞吐量、内存占用的巨大挑战。原生 FastAPI + 简单 WebSocket 难以支撑大规模并发。
2. **聊天历史数据混杂**
   文本消息、文件（图片、文档、表格）、结构化数据（如 Mermaid 图表代码）混存于同一数据表，导致查询效率低、文件管理困难、不支持按类型检索。
3. **客户端图表渲染能力缺失**
   Agent 返回的 Mermaid、UML、流程图等可视化内容无法在 H5 端正常渲染，影响用户体验（如架构图、时序图、思维导图等办公场景高频内容）。
4. **多 Agent 协作通信复杂**
   企业办公中往往需要多个专用 Agent（如邮件管家、数据分析师、日程助手）协同完成任务，现有单 Agent 模式无法实现 Agent 间的消息传递、任务委托、结果聚合。

本题目标

构建一个高性能、可扩展的 WorkClaw 云端办公助手，重点解决以下四大核心问题：

- 基于 **Netty** 构建高性能 WebSocket 网关，支撑 ≥200 并发连接，消息平均延迟 ≤100ms。
- 设计 **聊天历史分类存储方案**，区分文本、文件、图表代码等类型，支持按类型检索。
- 实现 **客户端图表渲染引擎**，支持 Mermaid、UML、流程图等实时渲染。
- 实现 **Agent 间通信机制**，允许多个专用 Agent 协同完成复杂办公任务。

二、核心需求

1. 高性能 WebSocket 网关（基于 Netty）

需求描述：在"用户 APP ↔ 云端实例"之间引入 Netty 中间层，作为高性能 WebSocket 代理/路由器。后端 OpenClaw 实例无需直接处理海量连接，而是通过 Netty 网关进行消息转发。

核心要求：

- 使用 **Netty** 实现 WebSocket 服务端，支持长连接管理、心跳检测、断线重连。
- 支持 **消息路由**：根据 userId 或 instanceId 将消息转发至对应的 OpenClaw 容器实例。
- 支持 **连接池/实例池** 映射，避免频繁创建销毁连接。
- 提供 **性能监控指标**：当前连接数、消息吞吐量（TPS）、平均延迟、内存占用。

性能指标	目标值
最大并发连接数	≥200
消息平均延迟（P99）	≤100ms
单节点吞吐量	≥500 msg/s
连接建立时间	≤50ms

架构示意：

```
┌──────────┐          WebSocket          ┌──────────────────────────┐  gRPC/HTTP
┌──────────┐
│ H5/APP   │◄──────────────────────────►│   Netty WebSocket 网关   │──────────►│
OpenClaw   │
│ 客户端   │                             │ ┌──────────┐  ┌──────────┐ │           │
容器实例 A  │
└──────────┘                             │ │ 连接管理 │  │ 消息路由 │ │           └────
──────────┘
                                         │ │ 心跳检测 │  │用户-实例 │ │
                                         │ └──────────┘  │映射表    │ │  ┌──────────┐
                                         │ ┌──────────┐  └──────────┘ │  │ OpenClaw
│
                                         │ │ 监控指标 │  ┌──────────┐ │  │ 容器实例 B │
                                         │ └──────────┘  │ 协议适配 │ │  └──────────
」
                                         │               └──────────┘ │
                                         └──────────────────────────┘
```

2. 聊天历史分类存储

需求描述：重新设计聊天记录存储模型，区分**文本消息**、**文件消息**（图片、文档、表格等）、**图表代码消息**（Mermaid、UML、PlantUML 等），支持按类型快速检索和渲染。

存储要求：

- **文本消息**：直接存储纯文本内容。
- **文件消息**：文件存储至对象存储（如腾讯云 COS），数据库仅保存文件 URL、文件名、大小、类型。
- **图表代码消息**：存储原始代码（如 Mermaid 语法）及渲染所需的元数据（图表类型、版本）。
- 提供 API 支持：
  - 按会话获取所有文本消息
  - 按会话获取所有文件列表
  - 按会话获取所有图表代码

数据模型示例（SQL）：

```
-- 消息基础表
message_id, session_id, sender, timestamp, message_type (text/file/diagram)

-- 文本内容表（message_type=text）
message_id, content

-- 文件内容表（message_type=file）
message_id, file_url, file_name, file_size, mime_type

-- 图表内容表（message_type=diagram）
message_id, diagram_type (mermaid/uml/plantuml), raw_code, rendered_cache（可选）
```

3. 客户端图表渲染支持

需求描述：H5 客户端能够正确渲染 Agent 返回的 Mermaid、UML 等图表，无需用户跳转或下载。办公场景中常用图表包括：流程图、时序图、类图、甘特图、架构图等。

要求：

- 集成 Mermaid.js 或类似库，支持动态渲染。
- 对于 UML 图表，可选用 nomnoml 或 PlantUML（通过服务端转图片降级）。
- 图表代码应能一键复制、全屏查看、导出为图片。
- 支持图表与其他文本内容混排（如"下面是一个用户登录流程图：mermaid ..."）。

示例渲染效果：

```
用户：请画出用户领养小龙虾的时序图

AI：好的，下面是时序图：

```mermaid
sequenceDiagram
    participant 用户
    participant H5
    participant Netty网关
    participant 容器实例
    用户->>H5：点击领养
    H5->>Netty网关：WebSocket请求
    Netty网关->>容器实例：创建实例
    容器实例-->>Netty网关：就绪
    Netty网关-->>H5：实例信息
    H5-->>用户：孵化成功
```
```

4. Agent 间通信机制

需求描述：企业办公场景中，一个任务往往需要多个 Agent 协作完成。例如："帮我总结今天的邮件，并根据邮件中的会议安排创建日程"。需要**邮件管家 Agent**与**日程管理 Agent**协作。

实现要求：

- 支持 **Agent 注册与发现**：每个 OpenClaw 实例启动时向控制平面注册其能力（技能列表）。
- 支持 **任务委托**：Agent A 可将子任务发送给 Agent B，并等待结果返回。
- 支持 **消息路由**：控制平面根据任务类型将消息转发至对应的 Agent。
- 提供 **协作日志**：记录 Agent 间的交互过程，便于调试和展示。

简单协作流程示例：

```
用户 → 邮件管家 Agent：总结邮件并创建日程
邮件管家 Agent → 控制平面：请求日程创建能力
控制平面 → 日程管理 Agent：创建日程（时间、主题）
日程管理 Agent → 控制平面：返回创建结果
控制平面 → 邮件管家 Agent：日程已创建
邮件管家 Agent → 用户：已完成，日程已添加到日历
```

技术选型提示：可使用消息队列（Redis Stream / RabbitMQ）或基于 Netty 的内部通信协议。

三、交付要求

- ☐ 完整可运行的系统，包含 H5 前端、Netty WebSocket 网关、后端控制平面、至少 2 个 OpenClaw 容器实例（可用模拟或轻量级实现）。
- ☐ Netty 网关：提供连接数监控页面或 API，演示 ≥50 并发连接下的消息收发（可使用压测工具或浏览器多标签模拟）。
- ☐ 聊天历史分类存储：数据库表结构设计文档 + API 演示（按类型查询历史）。
- ☐ 客户端图表渲染：至少展示 2 种图表（如 Mermaid 流程图 + UML 类图）的正常渲染。
- ☐ Agent 间通信：实现一个跨 Agent 协作场景（如"邮件摘要 + 日程创建"），并提供可视化日志。

演示场景（至少包含以下三个）：

1. 用户通过 H5 与"邮件管家"对话，Agent 返回 Mermaid 格式的邮件处理流程图，客户端正常渲染。
2. 用户要求"总结邮件并创建日程"，触发邮件管家与日程管理 Agent 协作，最终返回日程创建成功。
3. 用户查看历史文件列表，展示之前上传的文档及对应分析结果。

- ☐ 技术架构文档：包含 Netty 网关设计、消息路由策略、存储模型、Agent 协作协议。
- ☐ 答辩 PPT：突出高并发解决方案、性能测试数据、Agent 协作流程。

四、评分标准

维度	权重	评分要点
技术实现	40%	Netty 网关是否稳定支持高并发（连接数、延迟）；聊天历史分类存储是否合理、API 是否完善；客户端图表渲染是否流畅、支持多种类型；Agent 间通信是否完整、日志可查
产品体验	25%	对话响应是否快速（网关带来的延迟感知）；图表渲染是否自然、交互是否友好；跨 Agent 协作是否对用户透明、结果准确
架构设计	20%	Netty 网关与后端实例的解耦程度；消息路由、协议适配的可扩展性；存储模型的可维护性；Agent 注册发现机制
创新性	15%	是否有独特的性能优化技巧（如连接复用、批量推送）；是否提供了超出要求的图表类型或协作模式；是否有优雅的降级/容错方案

五、开发建议与技术栈

层级	推荐技术
前端	Vue3 + Vant + Mermaid.js（图表渲染）
Netty 网关	Java + Netty（或可选 Vert.x / Spring WebFlux）
后端控制平面	Python FastAPI / Go Gin
数据库	PostgreSQL（JSONB 支持灵活存储）
对象存储	腾讯云 COS / MinIO（本地模拟）
Agent 协作	Redis Stream / 内存消息队列
容器编排	Docker Compose（本地模拟多实例）

注：若团队对 Java/Netty 不熟悉，也可选用 Node.js + Socket.IO 或 Go + Gorilla WebSocket，但需自行评估并发性能。

六、风险提示与降级方案

风险	应对方案
Netty 学习曲线陡峭	Day1 完成简单 Echo 服务器；使用 Netty 官方 example 改造
客户端图表渲染兼容性	备选方案：服务端将 Mermaid 转为 PNG 图片下发给客户端
Agent 间通信过于复杂	先实现最简单的"请求-响应"模式，不要求异步回调
性能压测环境不足	使用 JMeter 或 wrk 进行本地压测，提供理论计算数据